

From Wikipedia, the free encyclopedia

This article is about the technical specifics and history of the PDF document format. For other uses, see [PDF \(disambiguation\)](#).

Portable Document Format



Adobe PDF icon

[Filename extension](#) .pdf

[Internet media type](#)

- [application/pdf](#),^[1]
- [application/x-pdf](#)
- [application/x-bzpdf](#)
- [application/x-gzpdf](#)

[Type code](#) PDF ^[1] (including a single trailing space)

[Uniform Type Identifier \(UTI\)](#) com.adobe.pdf

[Magic number](#) %PDF

Developed by [Adobe Inc.](#) (1991–2008)
 [ISO](#) (2008–)

Initial release June 15, 1993; 32 years ago

[Latest release](#) 2.0

Extended to [PDF/A](#), [PDF/E](#), [PDF/UA](#), [PDF/VT](#), [PDF/X](#)

Standard	ISO 32000-2
Open format?	Yes
Website	iso.org/standard/75839.html

Portable Document Format (PDF), standardized as **ISO 32000**, is a [file format](#) developed by [Adobe](#) in 1993 used to present [documents](#), including text formatting and images, in a manner independent of [application software](#), [hardware](#), and [operating systems](#).^{[2][3]} Based on the [PostScript](#) language, each PDF file encapsulates a complete description of a fixed-layout flat document, including the text, [fonts](#), [vector graphics](#), [raster images](#) and other information needed to display it. PDF has its roots in "The Camelot Project" initiated by Adobe co-founder [John Warnock](#) in 1991.^[4] PDF was standardized as ISO 32000 in 2008.^[5] It is maintained by ISO TC 171 SC 2 WG8, of which the [PDF Association](#) is the committee manager.^[6] The last edition as ISO 32000-2:2020 was published in December 2020.^[7]

PDF files may contain a variety of content besides flat text and graphics including logical structuring elements, interactive elements such as annotations and form-fields, layers, [rich media](#) (including video content), three-dimensional objects using [U3D](#) or [PRC](#), and various other [data formats](#). The PDF specification also provides for encryption and [digital signatures](#), file attachments, and [metadata](#) to enable [workflows](#) requiring these features.

History

Main article: [History of PDF](#)

The development of PDF began in 1991 when [John Warnock](#) wrote a paper for a project then code-named Camelot, in which he proposed the creation of a simplified version of PostScript called Interchange PostScript (IPS).^[8] Unlike traditional PostScript, which was tightly focused on rendering [print jobs](#) to output devices, IPS would be optimized for displaying pages to any screen and any platform.^[8]

[Adobe Systems](#) made the PDF specification available free of charge in 1993. In the early years PDF was popular mainly in [desktop publishing](#) workflows, and competed with several other formats, including [DjVu](#), [Envoy](#), Common Ground Digital Paper, Farallon Replica and even Adobe's own PostScript format.

PDF was a [proprietary format](#) controlled by Adobe until it was released as an [open standard](#) on July 1, 2008, and published by the [International Organization for Standardization](#) as ISO 32000-1:2008,^{[9][10]} at which time control of the specification passed to an ISO Committee of volunteer industry experts. In 2008, Adobe published a Public Patent License to ISO 32000-1 granting [royalty-free](#) rights for all patents owned by Adobe necessary to make, use, sell, and distribute PDF-compliant implementations.^[11]

PDF 1.7, the sixth edition of the PDF specification that became ISO 32000-1, includes some proprietary technologies defined only by Adobe, such as [Adobe XML Forms Architecture](#) (XFA) and [JavaScript](#) extension for Acrobat, which are referenced by ISO 32000-1 as [normative](#) and indispensable for the full implementation of the ISO 32000-1 specification.^[12] These proprietary technologies are not standardized, and their specification is published only on Adobe's website.^{[13][14][15]} Many of them are not supported by popular third-party implementations of PDF.

ISO published version 2.0 of PDF, ISO 32000-2 in 2017, available for purchase, replacing the free specification provided by Adobe.^[16] In December 2020, the second edition of PDF 2.0, ISO 32000-2:2020, was published, with clarifications, corrections, and critical updates to normative references^[17] (ISO 32000-2 does not include any proprietary technologies as normative references).^[18] In April 2023 the PDF Association made ISO 32000-2 available for download free of charge.^[16]

Technical details

A PDF file is often a combination of [vector graphics](#), text, and [bitmap graphics](#). The basic types of content in a PDF are:

- Typeset text stored as content streams (i.e., not encoded in [plain text](#));
- Vector graphics for illustrations and designs that consist of shapes and lines;
- Raster graphics for photographs and other types of images; and
- Other multimedia objects.

In later PDF revisions, a PDF document can also support links (inside document or web page), forms, JavaScript (initially available as a plugin for Acrobat 3.0), or any other types of embedded contents that can be handled using plug-ins.

PDF combines three technologies:

- An equivalent subset of the PostScript page description programming language but in declarative form, for generating the layout and graphics.
- A [font-embedding](#)/replacement system to allow fonts to travel with the documents.
- A structured storage system to bundle these elements and any associated content into a single file, with [data compression](#) where appropriate.

PostScript language

[PostScript](#) is a [page description language](#) run in an [interpreter](#) to generate an image.^[8] It can handle graphics and has standard features of [programming languages](#) such as [branching](#) and [looping](#).^[8] PDF is a subset of PostScript, simplified to remove such [control flow](#) features, while graphics commands remain.^[8]

PostScript was originally designed for a drastically different [use case](#): transmission of one-way linear print jobs in which the PostScript interpreter would collect a series of commands until it encountered the showpage command, then execute all the commands to render a page as a raster image to a printing device.^[19] PostScript was not intended for long-term storage and real-time interactive rendering of [electronic documents](#) to [computer monitors](#), so there was no need to support anything other than consecutive rendering of pages.^[19] If there was an error in the final printed output, the user would correct it at the application level and send a new print job in the form of an entirely new PostScript file. Thus, any given page in a PostScript file could be accurately rendered only as the cumulative result of executing all preceding commands to draw all previous pages—any of which could affect subsequent pages—plus the commands to draw that particular page, and there was no easy way to bypass that process to skip around to different pages.^[19]

Traditionally, to go from PostScript to PDF, a source PostScript file (that is, an executable program) is used as the basis for generating PostScript-like PDF code (see, e.g., [Adobe Distiller](#)). This is done by applying standard [compiler](#) techniques like [loop unrolling](#), [inlining](#) and removing unused branches, resulting in code that is purely declarative and static.^[19] The result is then packaged into a [container format](#), together with all necessary [dependencies](#) for correct rendering (external files, graphics, or fonts to which the document refers), and [compressed](#). Modern applications write to printer drivers that directly generate PDF rather than going through PostScript first.

As a document format, PDF has several advantages over PostScript:

- PDF contains only static [declarative](#) PostScript code that can be processed as data, and does not require a full program [interpreter](#) or [compiler](#).^[19] This avoids the complexity and security risks of an engine with such a higher complexity level.
- Like [Display PostScript](#), PDF has supported [transparent graphics](#) since version 1.4, while standard PostScript does not.
- PDF enforces the rule that the code for any particular page cannot affect any other pages.^[19] That rule is strongly recommended for PostScript code too, but has to be implemented explicitly (see, e.g., the [Document Structuring Conventions](#)), as PostScript is a full programming language that allows for such greater flexibilities and is not limited to the concepts of pages and documents.
- All data required for rendering is included within the file itself, improving portability.^[20]

Its disadvantages are:

- A loss of flexibility, and limitation to a single use case.^[citation needed]
- A (sometimes much) larger file size.^[21]

PDF since v1.6 supports embedding of interactive 3D documents: 3D drawings can be embedded using [U3D](#) or [PRC](#) and various other data formats.^{[22][23][24]}

File format

A PDF file is organized using [ASCII](#) characters, except for certain elements that may have binary content. The file starts with a header containing a [magic number](#) (as a readable string) and the version of the format, for example %PDF-1.7. The format is a subset of a COS ("Carousel" Object Structure) format.^[25] A COS tree file consists primarily of *objects*, of which there are nine types.^[18]

- [Boolean](#) values, representing *true* or *false*
- [Real numbers](#)
- [Integers](#)
- [Strings](#), enclosed within parentheses ((...)) or represented as hexadecimal within single angle brackets (<...>). Strings may contain 8-bit characters.
- Names, starting with a forward slash (/)
- [Arrays](#), ordered collections of objects enclosed within square brackets ([...])
- [Dictionaries](#), collections of objects indexed by names enclosed within double angle brackets (<<...>>)
- [Streams](#), usually containing large amounts of optionally compressed binary data, preceded by a dictionary and enclosed between the stream and endstream keywords.
- The [null](#) object

Comments using 8-bit characters prefixed with the percent sign (%) may be inserted.

Objects may be either *direct* (embedded in another object) or *indirect*. Indirect objects are numbered with an *object number* and a *generation number* and defined between the obj and endobj keywords if residing in the document root. Beginning with PDF version 1.5, indirect objects (except other streams) may also be located in special streams known as *object streams* (marked /Type /ObjStm). This technique enables non-stream objects to have standard stream filters applied to them, reduces the size of files that have large numbers of small indirect objects and is especially useful for *Tagged PDF*. Object streams do not support specifying an object's *generation number* (other than 0).

An index table, also called the cross-reference table, is located near the end of the file and gives the byte offset of each indirect object from the start of the file.^[26] This design allows for efficient [random access](#) to the objects in the file, and also allows for small changes to be made without rewriting the entire file (*incremental update*). Before PDF version 1.5, the

table would always be in a special ASCII format, be marked with the xref keyword, and follow the main body composed of indirect objects. Version 1.5 introduced optional *cross-reference streams*, which have the form of a standard stream object, possibly with filters applied. Such a stream may be used instead of the ASCII cross-reference table and contains the offsets and other information in binary format. The format is flexible in that it allows for integer width specification (using the /W array), so that for example, a document not exceeding 64 [KiB](#) in size may dedicate only 2 bytes for object offsets. To ensure backward compatibility, a hybrid-reference PDF file may include both traditional cross-reference tables and cross-reference streams, allowing older PDF processors to read the file while still taking advantage of the new features introduced in version 1.5.^[27]

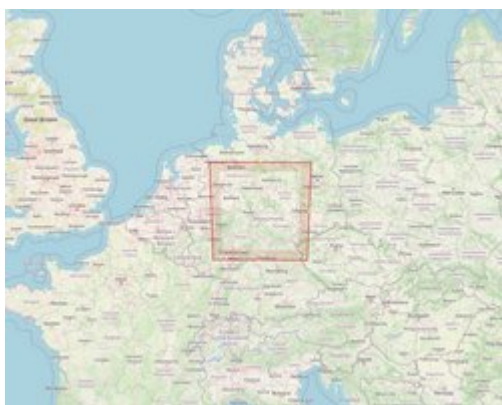
At the end of a PDF file is a footer containing

- The startxref keyword followed by an offset to the start of the cross-reference table (starting with the xref keyword) or the cross-reference stream object, followed by
- The %%EOF [end-of-file](#) marker.

If a cross-reference stream is not being used, the footer is preceded by the trailer keyword followed by a dictionary containing information that would otherwise be contained in the cross-reference stream object's dictionary:

- A reference to the root object of the tree structure, also known as the *catalog* (/Root)
- The count of indirect objects in the cross-reference table (/Size)
- Other optional information

Within each page, there are one or multiple content streams that describe the text, vector and images being drawn on the page. The content stream is [stack-based](#), similar to PostScript.^[28]



The maximum size of an Acrobat PDF page, superimposed on a map of Europe.

There are two layouts to the PDF files: non-linearized (not "optimized") and linearized ("optimized"). Non-linearized PDF files can be smaller than their linear counterparts, though

they are slower to access because portions of the data required to assemble pages of the document are scattered throughout the PDF file. Linearized PDF files (also called "optimized" or "web optimized" PDF files) are constructed in a manner that enables them to be read in a Web browser plugin without waiting for the entire file to download, since all objects required for the first page to display are optimally organized at the start of the file.^[29] PDF files may be optimized using [Adobe Acrobat](#) software or [QPDF](#).

Page dimensions are not limited by the format itself. However, Adobe Acrobat imposes a limit of 15 million by 15 million inches, or 225,000,000,000,000 square inches (145,161 km²; 56,047 sq mi), an area slightly larger than [Tajikistan](#).^{[2]: 1129}

Imaging model

The basic design of how graphics are represented in PDF is very similar to that of PostScript, except for the use of transparency, which was added in PDF 1.4.

PDF graphics use a [device-independent Cartesian coordinate system](#) to describe the surface of a page. A PDF page description can use a [matrix](#) to [scale](#), [rotate](#), or [skew](#) graphical elements. A key concept in PDF is that of the *graphics state*, which is a collection of graphical parameters that may be changed, saved, and restored by a *page description*. PDF has (as of version 2.0) 25 graphics state properties, of which some of the most important are:

- The *current transformation matrix* (CTM), which determines the coordinate system
- The [clipping path](#)
- The [color space](#)
- The [alpha constant](#), which is a key component of transparency
- [Black point compensation](#) control (introduced in PDF 2.0)

Vector graphics

As in PostScript, vector graphics in PDF are constructed with *paths*. Paths are usually composed of lines and cubic [Bézier curves](#), but can also be constructed from the outlines of text. Unlike PostScript, PDF does not allow a single path to mix text outlines with lines and curves. Paths can be stroked, filled, fill then stroked, or used for [clipping](#). Strokes and fills can use any color set in the graphics state, including *patterns*. PDF supports several types of patterns. The simplest is the *tiling pattern* in which a piece of artwork is specified to be drawn repeatedly. This may be a *colored tiling pattern*, with the colors specified in the pattern object, or an *uncolored tiling pattern*, which defers color specification to the time the pattern is drawn. Beginning with PDF 1.3 there is also a *shading pattern*, which draws continuously varying colors. There are seven types of shading patterns of which the simplest are the *axial shading* (Type 2) and *radial shading* (Type 3).

Raster images

Raster images in PDF (called *Image XObjects*) are represented by dictionaries with an associated stream. The dictionary describes the properties of the image, and the stream contains the image data. (Less commonly, small raster images may be embedded directly in a page description as an *inline image*.) Images are typically *filtered* for compression purposes. Image filters supported in PDF include the following general-purpose filters:

- *ASCII85Decode*, a filter used to put the stream into 7-bit ASCII,
- *ASCIIHexDecode*, similar to ASCII85Decode but less compact,
- *FlateDecode*, a commonly used filter based on the [deflate](#) algorithm defined in [RFC 1951](#) (deflate is also used in the [gzip](#), [PNG](#), and [zip](#) file formats among others); introduced in PDF 1.2; it can use one of two groups of predictor functions for more compact zlib/deflate compression: *Predictor 2* from the [TIFF 6.0](#) specification and predictors (filters) from the [PNG](#) specification ([RFC 2083](#)),
- *LZWDecode*, a filter based on [LZW](#) Compression; it can use one of two groups of predictor functions for more compact LZW compression: *Predictor 2* from the [TIFF 6.0](#) specification and predictors (filters) from the [PNG](#) specification,
- *RunLengthDecode*, a simple compression method for streams with repetitive data using the [run-length encoding](#) algorithm and the image-specific filters,
- *DCTDecode*, a [lossy](#) filter based on the [JPEG](#) standard,
- *CCITTFaxDecode*, a lossless [bi-level](#) (black/white) filter based on the Group 3 or [Group 4 CCITT](#) (ITU-T) [fax](#) compression standard defined in ITU-T [T.4](#) and T.6,
- *JBIG2Decode*, a lossy or [lossless](#) bi-level (black/white) filter based on the [JBIG2](#) standard, introduced in PDF 1.4, and
- *JPXDecode*, a lossy or lossless filter based on the [JPEG 2000](#) standard, introduced in PDF 1.5.

Normally all image content in a PDF is embedded in the file. But PDF allows image data to be stored in external files by the use of *external streams* or *Alternate Images*. Standardized subsets of PDF, including [PDF/A](#) and [PDF/X](#), prohibit these features.